

OPERATING SYSTEMS

HW 1 Code Document: SiSH

Saúl Gil. ID: 32239288

This assignment makes relation with the implementation of a SiSH program, this one acts as a command interpreter between the user and the operating system, this is going to be done with three main tools:

-fork: Creates a child process, trying to execute the specified command.

-execve: Execution of command.

-wait: Makes the father process to be blocked until the command finishes.

The code also has other characteristics such as the tokenization, dividing the text line in arguments, stop command (quit) or error management, if the user does a mistake or something incoherent, the system will notify it.

This code is written and run in Visual studio code. For the running, I use a wsl terminal and a gcc compiler, so you'll have to install the gcc extension if it's not installed yet.

First step is to compile the program and make sure there's no mistakes, I have done it with gcc -g sish.c ----(output name you prefer)

Compile the code you want to run (obviously skippable if you try to run a file, but I use a helloworld.c to see it clear, so I have to compile it).

Run the file generated

Type the specific location of the file you want to run, example:
/mnt/c/Users/*username*/(Folder)/*Name of the file*

Example with a helloworld file:

```
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$ gcc -g sish.c -o sish
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$ gcc -o helloworld helloworld.c
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$ ./sish
Shell SiSH. To exit, type quit
sish> /mnt/c/Users/usuario/Documents/helloworld
Hello world
sish> 
```

I don't know exactly why it does and if there's a loss of performance or something but I think it's easier to work with a execve, because you get rid of an external variable and may result confusing, so it would be interesting to check that in a future.

No freedays used yet.